BỘ KHOA HỌC VÀ CÔNG NGHỆ

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



BÁO CÁO BÀI TẬP LỚN

MÔN LẬP TRÌNH MẠNG

HỆ THỐNG CHẤM BÀI LẬP TRÌNH TỰ ĐỘNG DỰA TRÊN KIẾN TRÚC TCP SOCKET VÀ HÀNG ĐỢI XỬ LÝ ĐA LUỒNG

Người hướng dẫn : MSc. Ngô Văn Quyền

Họ và tên sinh viên : Nguyễn Văn Hoàng - B23DCCN341

Phạm Văn Kiên - B23DCCN466

Đào Trung Hiếu - B23DCCN298

Chuyên ngành : Công nghệ phần mềm

Khoa : Công nghệ thông tin 1

Hà Nội, 10/2026

NHẬN XÉT CỦA GIẢNG VIÊN CHẤM BÀI

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Điểm số: .....

Điểm chữ: .....

Giảng viên (ký và ghi rõ họ tên): .....

MỤC LỤC

TÓM TẮT **5**
DANH MỤC THUẬT NGỮ VÀ CHỮ VIẾT TẮT **6**
DANH MỤC HÌNH ẢNH **7**
DANH MỤC BẢNG BIỂU **8**
I. TỔNG QUAN **9**
1.1. Bối cảnh và động lực nghiên cứu 9
1.1.1. Thực trạng giảng dạy lập trình và nhu cầu chấm tự động 9
1.1.2. Thách thức về tải đồng thời, an toàn thực thi và liêm chính học thuật 9
1.2. Phát biểu bài toán và phạm vi đề tài 9
1.2.1. Phát biểu bài toán 9
1.2.2. Phạm vi và giới hạn 10
1.3. Mục tiêu đề tài 10
1.4. Đóng góp chính 10
II. CÁC NGHIÊN CỨU LIÊN QUAN VÀ CƠ SỞ LÝ THUYẾT **12**
2.1. Cơ sở lý thuyết truyền thông mạng và xử lý đồng thời 12
2.2. Khảo sát các hệ thống chấm mã nguồn hiện nay 12
2.2.1. Các nền tảng quốc tế: Codeforces và LeetCode 12
2.2.2. Hệ thống thực tế tại Học viện: Code PTIT 12
2.3. Thảo luận về những hạn chế trong các hệ thống đã có 13
2.3.1. Bài toán hướng đối tượng 13
2.3.2. Lãng phí tài nguyên và độ trễ từ cơ chế thăm dò HTTP 13
2.3.3. Lỗ hổng giám sát trước sự bùng nổ của AI 13
2.4. Đóng góp mới của hệ thống 14
III. PHÂN TÍCH YÊU CẦU VÀ THIẾT KẾ HỆ THỐNG **15**
3.1. Định nghĩa bài toán và yêu cầu hệ thống 15
3.1.1. Dữ liệu đầu vào, đầu ra và ràng buộc kỹ thuật 15
3.1.2. Yêu cầu chức năng và phi chức năng 15
3.2. Kiến trúc tổng thể hệ thống 15
3.2.2. Mô hình phân tầng 15
3.2.3. Trách nhiệm các phân hệ và quy trình chấm hai pha 16
3.3. Thiết kế truyền thông mạng 17
3.3.1. Đóng khung dữ liệu và cấu trúc thông điệp 17
3.3.2. Tập mã lệnh điều khiển 17
3.3.3. Trình tự trao đổi thông điệp 18
3.3.4. Duy trì phiên và xử lý đứt kết nối 18
3.4. Thiết kế phân hệ xử lý đồng thời và hàng đợi chấm bài 19
3.4.1. Kiến trúc hàng đợi LinkedBlockingQueue kết hợp Worker Pool 19

3.4.2. Luận cứ sử dụng hàng đợi nội bộ thay cho Message Broker 19
3.4.3. Cơ chế kiểm soát tràn bộ đệm Backpressure..... 19
3.5. Thiết kế lưu trữ dữ liệu và an toàn hệ thống 20
3.5.1. Thiết kế cơ sở dữ liệu quan hệ..... 20
3.5.2. Phân tầng lưu trữ tệp nguồn qua kho đối tượng MinIO..... 21
3.5.3. Cơ chế phòng chống gian lận và bảo vệ kỳ thi..... 21
IV. TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG 23
4.1. Triển khai các thành phần phần mềm cốt lõi 23
4.1.1. Máy chủ socket đa luồng và cơ chế thông báo đẩy 23
4.1.2. Ứng dụng khách JavaFX và giao diện tương tác thời gian thực 23
4.1.3. Động cơ chấm OOP hai pha và cơ chế cô lập an toàn..... 25
4.2. Thiết lập môi trường và kịch bản thực nghiệm 26
4.2.1. Môi trường thực nghiệm và bộ chỉ số đo lường hiệu năng..... 26
4.2.2. Thiết kế hai bộ thực nghiệm kiểm soát biến độc lập 26
4.3. Kết quả thực nghiệm và đo lường định lượng 27
4.3.1. Hiệu năng mở rộng theo số lượng luồng chấm 27
4.3.2. Năng lực chịu tải theo quy mô bài nộp và độ ổn định hàng đợi..... 27
4.3.3. Đánh giá độ chính xác và khả năng xử lý bài nộp đặc thù..... 28
4.4. Phân tích và đánh giá kỹ thuật 29
4.4.1. Đánh giá giữa chi phí tài nguyên và mức độ cách ly an toàn..... 29
4.4.2. Đánh giá giữa đơn giản hóa kiến trúc mạng và khả năng mở rộng phân tán..... 29
V. KẾT LUẬN 30
5.1. Tổng kết kết quả đạt được..... 30
5.2. Bảng tổng hợp đóng góp kỹ thuật và bằng chứng thực nghiệm 30
5.3. Những hạn chế tồn tại của hệ thống 31
5.4. Hướng nghiên cứu và phát triển..... 31
TÀI LIỆU THAM KHẢO..... 32

TÓM TẮT

Báo cáo trình bày thiết kế và cài đặt hệ thống chấm mã nguồn tự động theo mô hình máy khách - máy chủ sử dụng socket TCP ổn định. Nhằm giải quyết điểm nghẽn của mô hình tuần tự và loại bỏ chi phí băng thông từ kỹ thuật thăm dò HTTP polling, hệ thống kết hợp hàng đợi trong bộ nhớ với nhóm worker xử lý song song. Bên cạnh việc đánh giá thuật toán qua luồng nhập xuất chuẩn, hệ thống tích hợp máy chấm hướng đối tượng hai pha dựa trên Java Reflection để kiểm tra trực tiếp cấu trúc lớp và gọi phương thức độc lập. Tiến trình chấm được bảo vệ nhờ bộ quét mã độc và cơ chế cô lập tiến trình có giới hạn tài nguyên. Máy chủ chủ động trả kết quả về máy khách theo thời gian thực ngay khi hoàn tất. Thực nghiệm xác nhận hệ thống nâng thông lượng xử lý gấp hơn ba lần mô hình cơ sở, giữ được độ trễ ổn định ngay cả khi tải cao và loại bỏ hoàn toàn lưu lượng mạng dư thừa.

DANH MỤC THUẬT NGỮ VÀ CHỮ VIẾT TẮT

Thuật ngữ	**Ý nghĩa trong nghiên cứu**
Giao thức TCP	Giao thức tầng giao vận đảm bảo truyền tải mã nguồn và kết quả chấm tin cậy, nguyên vẹn và đúng thứ tự.
Thông điệp JSON	Định dạng giao thức tầng ứng dụng (JSON phân cách dòng \n) dùng để trao đổi dữ liệu giữa Client và Server.
Hàng đợi Worker	Kiến trúc bất đồng bộ BlockingQueue + Worker Pool giúp nhận bài nộp tức thì và xử lý chấm song song, tránh nghẽn hệ thống.
Kiểm thử hướng đối tượng	Ứng dụng kỹ thuật Reflection để kiểm tra tính đúng đắn của cấu trúc OOP (tính đóng gói, hàm khởi tạo, getter/setter) trước khi chấm.
Java Reflection	Kỹ thuật kiểm thử động cấu trúc hướng đối tượng (tính đóng gói, constructor, getter/setter…) của bài nộp trước khi đưa vào chấm test case.

DANH MỤC HÌNH ẢNH

Hình 3.1: Sơ đồ kiến trúc phân tầng tổng thể hệ thống 15
Hình 3.2: Sơ đồ quy trình động cơ chấm lập trình hướng đối tượng hai pha 16
Hình 3.3: Biểu đồ tuần tự trao đổi giao thức mạng giữa máy khách và máy chủ 17
Hình 3.4: Lược đồ quan hệ thực thể cơ sở dữ liệu hệ thống 19
Hình 4.1: Giao diện sinh viên nộp bài và nhận kết quả phán quyết thời gian thực qua socket TCP 23
Hình 4.2: Giao diện giảng viên giám sát phòng thi và cảnh báo vi phạm trực tiếp.. 24

DANH MỤC BẢNG BIỂU

Bảng 4.1: Ảnh hưởng của số lượng worker đến hiệu năng xử lý 26
Bảng 4.2: Ảnh hưởng của quy mô tải bài nộp khi cố định cấu hình máy chủ 27
Bảng 5.1: Tổng hợp đóng góp kỹ thuật cốt lõi và bằng chứng thực nghiệm 29

I. TỔNG QUAN

1.1. Bối cảnh và động lực nghiên cứu

1.1.1. Thực trạng giảng dạy lập trình và nhu cầu chấm tự động

Đào tạo lập trình đại học đòi hỏi việc đánh giá bài nộp phải chính xác và phản hồi tức thì. Trong các kỳ thi tại phòng máy, hàng trăm máy trạm đồng thời gửi mã nguồn khiến việc chấm thủ công bất khả thi. Các hệ thống web hiện nay dùng giao thức HTTP một chiều buộc máy khách liên tục gửi gói tin thăm dò tiến độ. Điều này không chỉ làm tốn băng thông, mà còn khiến máy chủ phải xử lý thêm các gói tin lặp đi lặp lại. Kết quả là phản hồi tới sinh viên bị chậm và hiệu quả cũng giảm đi nhiều.

1.1.2. Thách thức về tải đồng thời, an toàn thực thi và liêm chính học thuật

Khi số lượng kết nối tăng đột ngột, máy chủ xử lý tuần tự dễ nghẽn tắc nếu gặp bài nộp chứa vòng lặp vô hạn. Mặt khác, việc chạy trực tiếp mã nguồn sinh viên tiềm ẩn nguy cơ an ninh như lệnh xóa tệp hệ thống hoặc mở socket trái phép. Hệ thống chấm mạng do đó phải phân phối bài nộp bất đồng bộ mượt mà, đồng thời thiết lập cơ chế cô lập an toàn tiến trình chấm mà không lãng phí tài nguyên máy chủ.

Đặc biệt, sự phổ biến của các công cụ sinh mã tự động bằng trí tuệ nhân tạo đặt ra thách thức cấp bách trong việc giám sát hành vi làm bài của thí sinh tại máy trạm nhằm đảm bảo tính công bằng và liêm chính trong đánh giá học thuật.

1.2. Phát biểu bài toán và phạm vi đề tài

1.2.1. Phát biểu bài toán

Đề tài thiết kế và cài đặt hệ thống chấm mã nguồn tự động phân tán theo mô hình client - server qua giao thức TCP. Hệ thống tiếp nhận bài nộp từ ứng dụng giao diện đồ họa, đưa vào hàng đợi điều phối, chấm độc lập và tự động đẩy kết quả về máy khách theo thời gian thực. Yêu cầu then chốt gồm đóng khung gói tin trên luồng byte TCP, tách biệt luồng mạng với luồng chấm, kiểm thử hướng đối tượng chuyên sâu và ngăn chặn mã độc phá hoại máy chủ.

1.2.2. Phạm vi và giới hạn

Hệ thống triển khai qua socket TCP trên cổng 9999, hỗ trợ bốn ngôn ngữ Java, C, C++, Python và trọng tâm là Java OOP. Client sử dụng đồ họa JavaFX, kết nối máy chủ đa luồng tích hợp cơ sở dữ liệu PostgreSQL và kho lưu trữ MinIO. Đề tài tối ưu kiến trúc socket nội bộ, không sử dụng message broker ngoài nhằm giữ hạ tầng gọn nhẹ và độc lập.

1.3. Mục tiêu đề tài

Đề tài xác định ba mục tiêu kỹ thuật cụ thể.

- Thứ nhất, thiết kế giao thức mạng trên socket TCP bền vững dùng định dạng JSON phân dòng để truyền nhận hai chiều và đẩy kết quả thời gian thực, loại bỏ hoàn toàn thăm dò HTTP.
- Thứ hai, xây dựng kiến trúc xử lý đồng thời với hàng đợi chặn nội bộ và nhóm luồng worker song song, tích hợp cơ chế điều tiết dòng tải nhằm tối ưu thông lượng và ngừa tràn bộ nhớ.
- Cuối cùng, phát triển động cơ chấm hướng đối tượng chuyên dụng bằng phản xạ động hai pha, đánh giá trực tiếp cấu trúc lớp và hành vi phương thức mà không phụ thuộc luồng nhập xuất chuẩn, đi kèm cơ chế cô lập tiến trình an toàn.

1.4. Đóng góp chính

Đề tài mang lại bốn đóng góp kỹ thuật thực tế cho hệ thống chấm mã nguồn tự động qua mạng:

- Thiết kế giao thức Delimited JSON trên socket TCP bền vững: Chuẩn hóa tập mã lệnh ngắn gọn, tích hợp xác thực phiên và cơ chế đẩy kết quả hai chiều thời gian thực, loại bỏ hoàn toàn chi phí thăm dò HTTP.
- Triển khai hàng đợi chặn nội bộ kết hợp worker chấm song song: Phân tách rạch ròi luồng mạng với luồng chấm giúp máy chủ phản hồi tức thì, triệt tiêu sự phụ thuộc vào các message broker ngoài.

- Phát triển động cơ chấm hướng đối tượng: Thẩm định trực tiếp cấu trúc lớp và kích hoạt phương thức tại thời điểm chạy không phụ thuộc console, giải quyết trọn vẹn bài toán chấm tự động bài tập hướng đối tượng.
- Xây dựng giải pháp phòng thủ an toàn và kịch bản thực nghiệm định lượng: Kết hợp bộ lọc tĩnh mã độc với cơ chế giới hạn tài nguyên tiến trình, đi kèm kịch bản đo lường tự động cung cấp đầy đủ số liệu về thông lượng, độ trễ và mức tiêu thụ phần cứng.

II. CÁC NGHIÊN CỨU LIÊN QUAN VÀ CƠ SỞ LÝ THUYẾT

2.1. Cơ sở lý thuyết truyền thông mạng và xử lý đồng thời

Giao thức TCP cung cấp kênh truyền tin cậy, hướng kết nối và bảo toàn thứ tự dữ liệu trên nền IP. Do TCP truyền theo dòng byte liên tục, hiện tượng dính gói và phân mảnh luôn xảy ra khi gửi thông điệp liên tiếp. Kỹ thuật phân tách bằng ký tự ngắt dòng kết hợp định dạng JSON cho phép bên nhận bóc tách chính xác từng gói tin có cấu trúc.

Về xử lý đồng thời, việc tạo luồng mới cho mỗi kết nối dễ gây cạn kiệt bộ nhớ và quá tải chu kỳ CPU. Nhóm luồng định kích thước kết hợp hàng đợi chặn giúp đệm yêu cầu và phân phối tới worker rỗi, duy trì tài nguyên ổn định dưới áp lực tải cao.

2.2. Khảo sát các hệ thống chấm mã nguồn hiện nay

2.2.1. Các nền tảng quốc tế: Codeforces và LeetCode

Các nền tảng chấm trực tuyến quốc tế được tối ưu theo mục tiêu riêng. Codeforces phục vụ thi đấu, nạp dữ liệu vào đầu vào chuẩn và so khớp ký tự đầu ra với đáp án mẫu, chỉ hiệu quả cho giải thuật tuần tự mà không hỗ trợ kiểm thử đối tượng. LeetCode phục vụ luyện tập phỏng vấn qua các khung hàm gọi trực tiếp phương thức. Quá trình nộp bài trên web dùng giao thức HTTP, buộc máy khách liên tục thăm dò định kỳ để lấy trạng thái chấm.

2.2.2. Hệ thống thực tế tại Học viện: Code PTIT

Tại Học viện Công nghệ Bưu chính Viễn thông, Code PTIT là nền tảng chấm chính thức cho các môn lập trình cơ sở. Hệ thống vận hành theo nguyên lý so khớp luồng nhập xuất chuẩn: sinh viên viết hàm main tự đọc dữ liệu và in kết quả ra màn hình để đối chiếu với đáp án mẫu.

2.3. Thảo luận về những hạn chế trong các hệ thống đã có

Mặc dù những hệ thống nêu trên đã đáp ứng hầu hết được các chức năng luyện tập, kiểm tra và đánh giá kỹ năng lập trình, tuy nhiên, nó vẫn có những hạn chế nhất định do được xây dựng và phát triển dựa trên các mục tiêu sử dụng riêng biệt.

2.3.1. Bài toán hướng đối tượng

Mô hình so khớp luồng nhập xuất bất lực trước các bài toán hướng đối tượng vốn coi trọng tính bao gói, kế thừa và đa hình. Người học có thể viết theo lối thủ tục dồn vào hàm main, khai báo thuộc tính công khai hoặc dùng biến toàn cục nhưng vẫn in ra kết quả trùng khớp mẫu. Máy chấm nhập xuất sẽ cho điểm tuyệt đối dù bài làm vi phạm nguyên lý thiết kế. Việc kiểm tra hàm khởi tạo, quy tắc tự sinh mã hay ràng buộc thuộc tính cũng nằm ngoài khả năng so khớp chuỗi. Hệ quả là Code PTIT không thể chấm bài tập hướng đối tượng, buộc giảng viên phải chấm tay thủ công hàng chục giờ mỗi lớp, làm mất tính phản hồi tức thì.

2.3.2. Lãng phí tài nguyên và độ trễ từ cơ chế thăm dò HTTP

Hệ thống chấm web phụ thuộc mô hình hỏi đáp một chiều của HTTP. Trong thời gian bài làm xếp hàng chờ chấm, máy khách liên tục phát gói tin thăm dò trạng thái. Với phòng máy hàng trăm sinh viên, lượng yêu cầu HTTP gửi lên máy chủ lên tới hàng nghìn lượt mỗi phút nhưng hầu hết chỉ nhận về thông báo đang chờ. Sự dư thừa này gây nghẽn mạng nội bộ, tiêu hao năng lực xử lý tiêu đề máy chủ và tạo độ trễ nhận kết quả giữa các chu kỳ thăm dò.

2.3.3. Lỗ hổng giám sát trước sự bùng nổ của AI

Các hệ thống chấm bài hiện nay gần như bỏ qua khâu giám sát môi trường làm bài phía client. Sự phổ biến của các mô hình ngôn ngữ lớn (như ChatGPT, Claude, Gemini) cùng các trình soạn thảo tích hợp AI khiến tình trạng gian lận trở nên tinh vi và khó kiểm soát. Sinh viên có thể dễ dàng chuyển đổi cửa sổ, dùng ứng dụng phụ trợ hoặc sao chép - dán trực tiếp toàn bộ khối mã nguồn do AI sinh ra vào giao diện nộp bài. Máy chấm truyền thống chỉ nhìn thấy đoạn mã cuối cùng mà không thể truy

vết ngữ cảnh thao tác, không nhận diện được các dấu vết định dạng đặc trưng của AI. Điều này làm mất đi sự công bằng trong các kỳ thi lập trình tập trung và đặt gánh nặng giám thị lên việc kiểm tra thủ công từng màn hình máy tính.

2.4. Đóng góp mới của hệ thống

Với các điểm nghẽn kỹ thuật nêu trên, nhóm đề tài đã đưa ra giải pháp để có thể khắc phục được những hạn chế, bất cập, cụ thể như sau:

- Về truyền thông mạng, hệ thống thay thế cơ chế thăm dò HTTP bằng kết nối socket TCP bền vững hai chiều, cho phép máy chủ chủ động đẩy kết quả về máy khách theo thời gian thực ngay khi chấm xong, triệt tiêu lưu lượng mạng vô ích.
- Về nghiệp vụ đánh giá, hệ thống phát triển động cơ phản xạ động hai pha cho ngôn ngữ Java. Hệ thống kiểm tra trực tiếp bytecode sau biên dịch: soi chiếu phạm vi truy cập, kiểm tra chữ ký phương thức và kích hoạt kiểm thử đối tượng sống trong bộ nhớ mà không cần hàm main, tự động hóa hoàn toàn bài tập hướng đối tượng và giải phóng công sức chấm tay của giảng viên.
- Về giám sát thi và phòng chống gian lận, hệ thống tích hợp module phát hiện bất thường đa tầng tại máy khách kết hợp thông báo khẩn cho giám thị. Hệ thống quét định kỳ các tiến trình công cụ AI độc lập đang chạy nền, theo dõi biến động cửa sổ và phân tích mã nguồn để bắt các cú pháp do AI sinh ra. Mọi vi phạm được đóng gói thành sự kiện mạng và đẩy thẳng lên bảng giám sát của giảng viên theo thời gian thực, đảm bảo tính công bằng và minh bạch trong phòng thi.

III. PHÂN TÍCH YÊU CẦU VÀ THIẾT KẾ HỆ THỐNG

3.1. Định nghĩa bài toán và yêu cầu hệ thống

3.1.1. Dữ liệu đầu vào, đầu ra và ràng buộc kỹ thuật

Đầu vào hệ thống gồm thông tin xác thực, mã nguồn đơn hoặc gói ZIP, mã bài tập và cấu hình kiểm thử. Với bài tập hướng đối tượng, đầu vào bổ sung đặc tả JSON về thuộc tính, hàm khởi tạo và phương thức. Đầu ra là gói tin kết quả gồm phán quyết, điểm số, thời gian chạy, mức chiếm bộ nhớ và nhật ký chi tiết.

Hệ thống áp ba ràng buộc kỹ thuật: thời gian chạy tối đa mỗi ca kiểm thử hai giây; bộ nhớ cấp phát mỗi tiến trình con tối đa 256 megabyte; toàn bộ giao tiếp diễn ra trên cổng socket TCP 9999 duy nhất, không dùng máy chủ web trung gian.

3.1.2. Yêu cầu chức năng và phi chức năng

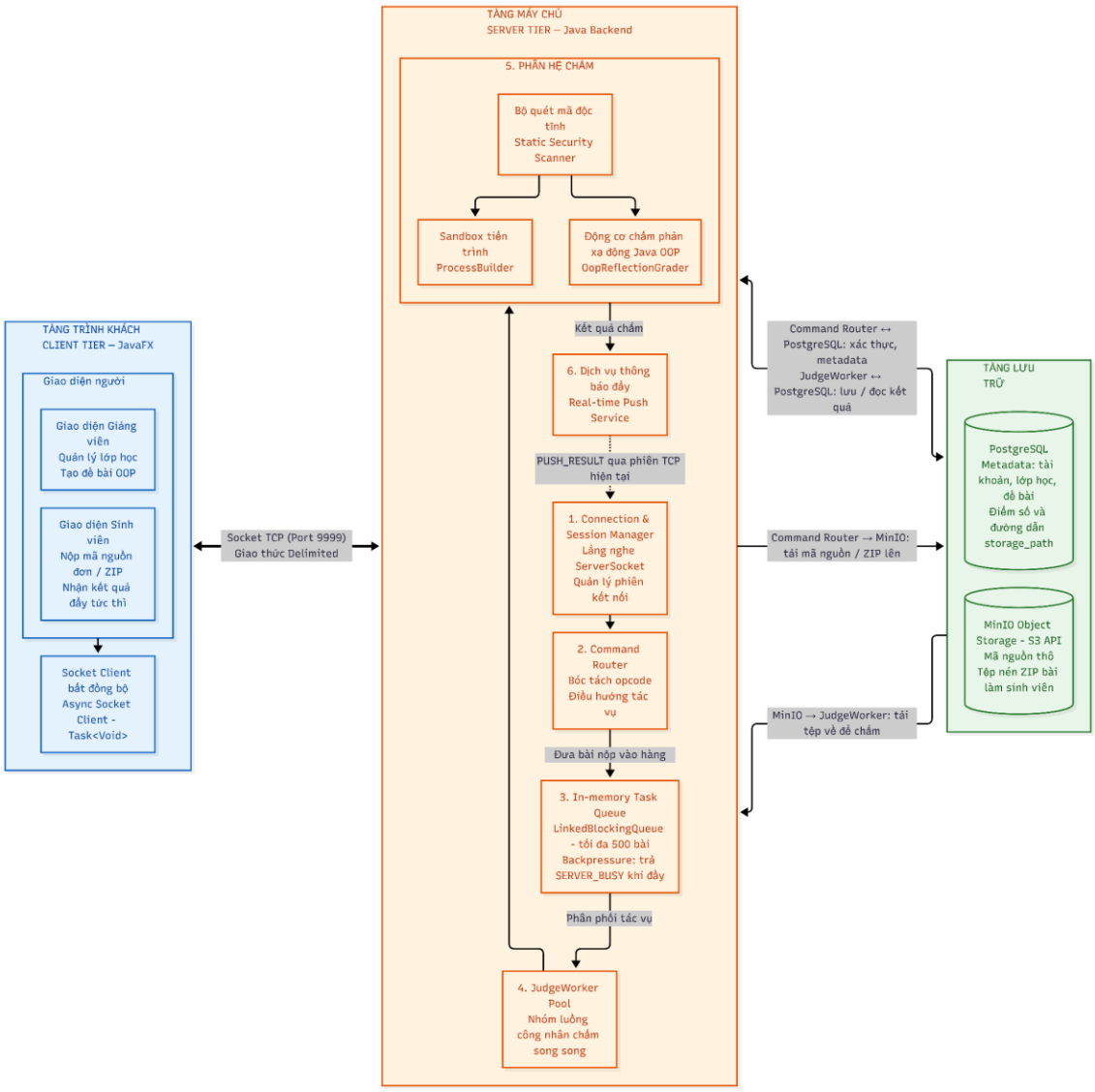
Về chức năng, giảng viên quản trị lớp tín chỉ, cấu hình bài tập, theo dõi bảng điểm trực tiếp; sinh viên nộp mã nguồn và nhận kết quả tức thì không cần tải lại trang.

Về phi chức năng, thời gian phản hồi xác nhận nộp bài dưới 200 mili-giây và độ trễ đẩy kết quả về máy khách dưới 100 mili-giây. Hệ thống tự khôi phục kết nối, ngăn mã độc can thiệp hệ điều hành và bảo toàn điểm số.

3.2. Kiến trúc tổng thể hệ thống

3.2.2. Mô hình phân tầng

Hệ thống tổ chức theo kiến trúc ba tầng độc lập nhằm phân tách giao diện người dùng, xử lý nghiệp vụ và lưu trữ dữ liệu.



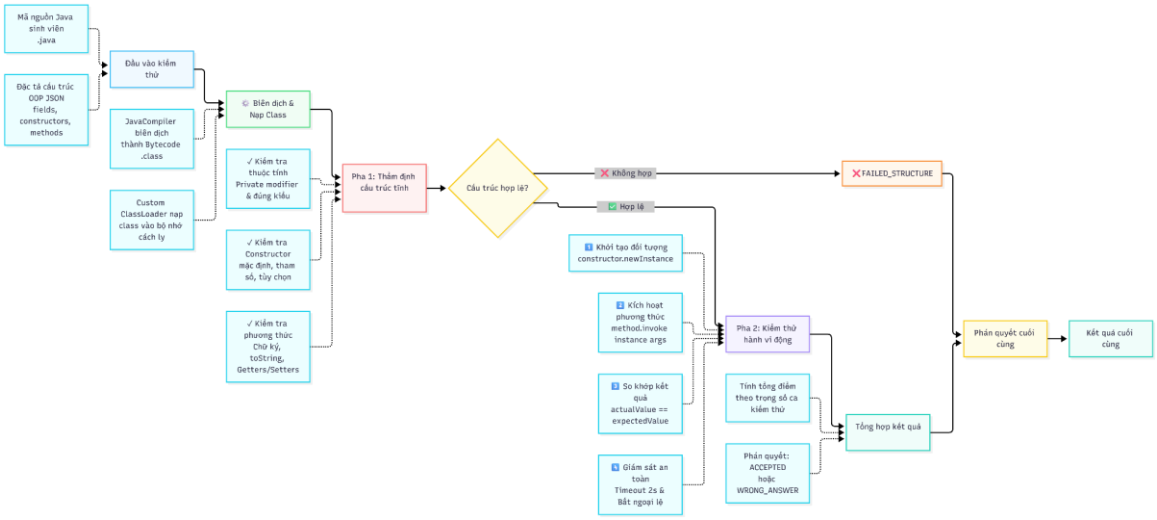
Hình 3.1: Sơ đồ kiến trúc phân tầng tổng thể hệ thống

Tầng trình khách chạy giao diện JavaFX trên máy trạm, duy trì kết nối socket TCP nền. Tầng máy chủ là tiến trình Java độc lập trên cổng 9999, tiếp nhận mạng, điều phối hàng đợi nội bộ và điều hành nhóm worker chấm song song. Tầng lưu trữ kết hợp cơ sở dữ liệu PostgreSQL quản lý dữ liệu nghiệp vụ và kho MinIO lưu trữ các gói mã nguồn.

3.2.3. Trách nhiệm các phân hệ và quy trình chấm hai pha

Phân hệ mạng gán mỗi kết nối socket cho một luồng, bóc tách khung JSON và chuyển sang bộ định tuyến. Phân hệ điều phối đưa bài nộp vào hàng đợi chặn trong

bộ nhớ rồi giải phóng ngay luồng mạng. Phân hệ động cơ chấm gồm các worker chạy ngầm, lấy bài từ hàng đợi, quét tĩnh mã độc và thực thi đánh giá.



Hình 3.2: Sơ đồ quy trình động cơ chấm lập trình hướng đối tượng hai pha

Quy trình chấm OOP diễn ra qua hai pha nối tiếp: Pha 1 kiểm tra cấu trúc nhị phân sau biên dịch bằng cơ chế phản xạ tĩnh; Pha 2 nạp động bytecode vào máy ảo Java cách ly để tạo đối tượng sống và kích hoạt kiểm thử hành vi phương thức. Khi hoàn tất, kết quả được lưu vào cơ sở dữ liệu và chuyển tới dịch vụ thông báo đẩy để trả về máy khách qua socket TCP mở sẵn.

3.3. Thiết kế truyền thông mạng

3.3.1. Đóng khung dữ liệu và cấu trúc thông điệp

Dữ liệu truyền qua TCP là dòng byte liên tục. Để chống dính gói và phân mảnh, hệ thống áp dụng kỹ thuật đóng khung dòng bằng ký tự ngắt dòng LF kết hợp định dạng JSON. Mỗi gói tin là một chuỗi JSON trọn vẹn trên một dòng đơn, kết thúc bằng LF. Cấu trúc gói chuẩn hóa gồm ba trường: opcode quy định loại lệnh nghiệp vụ, token xác thực quyền phiên làm việc và data chứa đối tượng payload chi tiết.

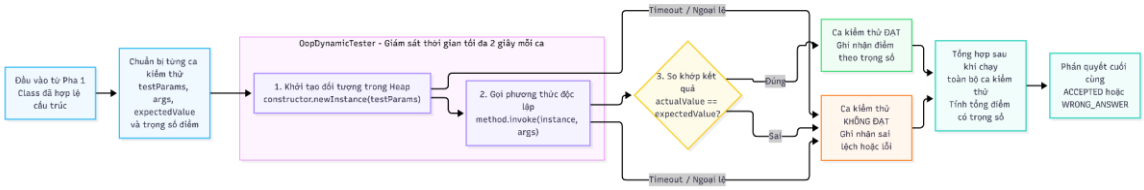
3.3.2. Tập mã lệnh điều khiển

Giao thức chuẩn hóa bốn nhóm mã lệnh chính: nhóm xác thực phiên tài khoản gồm AUTH_LOGIN, AUTH_LOGOUT và phản hồi AUTH_SUCCESS,

AUTH_FAILURE; nhóm quản lý đào tạo gồm GET_CLASSES, GET_PROBLEMS; nhóm kỳ thi và nộp bài gồm GET_ASSESSMENTS, START_ASSESSMENT, SUBMIT_CODE và xác nhận SUBMIT_ACK; nhóm thông báo đẩy gồm PUSH_RESULT và cảnh báo quá tải SERVER_BUSY. Chi tiết cấu trúc payload từng mã lệnh được tra cứu tại Phụ lục A.

3.3.3. Trình tự trao đổi thông điệp

Giao thức mạng vận hành theo hai chu trình trao đổi cốt lõi trên biểu đồ tuần tự.



Hình 3.3: Biểu đồ tuần tự trao đổi giao thức mạng giữa máy khách và máy chủ

Chu trình xác thực mở kết nối TCP tới cổng 9999, gửi mã AUTH_LOGIN đối chiếu mật khẩu băm và nhận token phiên. Các yêu cầu nghiệp vụ sau đó bắt buộc đính kèm token này trong gói tin để vượt qua bộ lọc an toàn.

Chu trình nộp bài diễn ra bất đồng bộ và không chặn luồng mạng: máy khách gửi SUBMIT_CODE, máy chủ nhận bài đưa vào hàng đợi và phản hồi ngay SUBMIT_ACK. Luồng mạng máy khách lập tức giải phóng. Khi worker chấm xong, dịch vụ thông báo đẩy chủ động gửi PUSH_RESULT qua socket TCP mở sẵn, giao diện tự cập nhật kết quả mà không cần máy khách gửi gói thăm dò.

3.3.4. Duy trì phiên và xử lý đứt kết nối

Máy chủ quản lý phiên bằng bảng liên kết mã người dùng với kênh socket tương ứng trong bộ nhớ. Khi máy trạm mất kết nối đột ngột, luồng đọc mạng lập tức đóng kênh socket, hủy đăng ký phiên và ghi nhật ký. Token phiên được lưu có thời hạn trong cơ sở dữ liệu giúp máy khách tự động tái xác thực khi có mạng trở lại. Kết quả

chấm hoàn thành trong lúc mất mạng vẫn được lưu bền vững và cập nhật ngay khi máy trạm kết nối lại.

3.4. Thiết kế phân hệ xử lý đồng thời và hàng đợi chấm bài

3.4.1. Kiến trúc hàng đợi LinkedBlockingQueue kết hợp Worker Pool

Nhằm tách biệt tốc độ nhận mạng tính bằng mili-giây với thời gian chấm bài kéo dài nhiều giây, máy chủ áp dụng mô hình hàng đợi LinkedBlockingQueue trong bộ nhớ và nhóm luồng worker cố định. Luồng mạng chỉ ghi nhận bài nộp, đóng gói và đưa vào hàng đợi. Luồng worker liên tục lấy bài để thực thi chấm. Nếu hàng đợi rỗng, worker tự động ngủ đông không tiêu hao chu kỳ xử lý; khi có bài mới, worker lập tức được đánh thức. Số luồng worker được ấn định tương ứng số lõi vi xử lý vật lý nhằm triệt tiêu chi phí chuyển đổi ngữ cảnh luồng.

3.4.2. Luận cứ sử dụng hàng đợi nội bộ thay cho Message Broker

Các hệ thống quy mô lớn thường dùng message broker phân tán như Kafka hoặc RabbitMQ. Tuy nhiên, trong hạ tầng này việc đưa vào các cụm trung gian này bộc lộ ba điểm bất hợp lý: phát sinh các dịch vụ điều phối phụ trợ phức tạp như Zookeeper gây khó bảo trì; tạo thêm hai chặng truyền thông nội bộ làm tăng độ trễ xử lý bài nộp; và tiêu tốn hàng gigabyte RAM duy trì dịch vụ nền trong khi bộ nhớ máy chủ cần ưu tiên cho biên dịch mã nguồn. Sử dụng LinkedBlockingQueue thuần túy trong bộ nhớ là giải pháp tối ưu: loại bỏ hoàn toàn độ trễ mạng phụ, giảm chiếm dụng RAM về mức tối thiểu và đảm bảo tính khép kín, tự vận hành độc lập của hệ thống.

3.4.3. Cơ chế kiểm soát tràn bộ đệm Backpressure

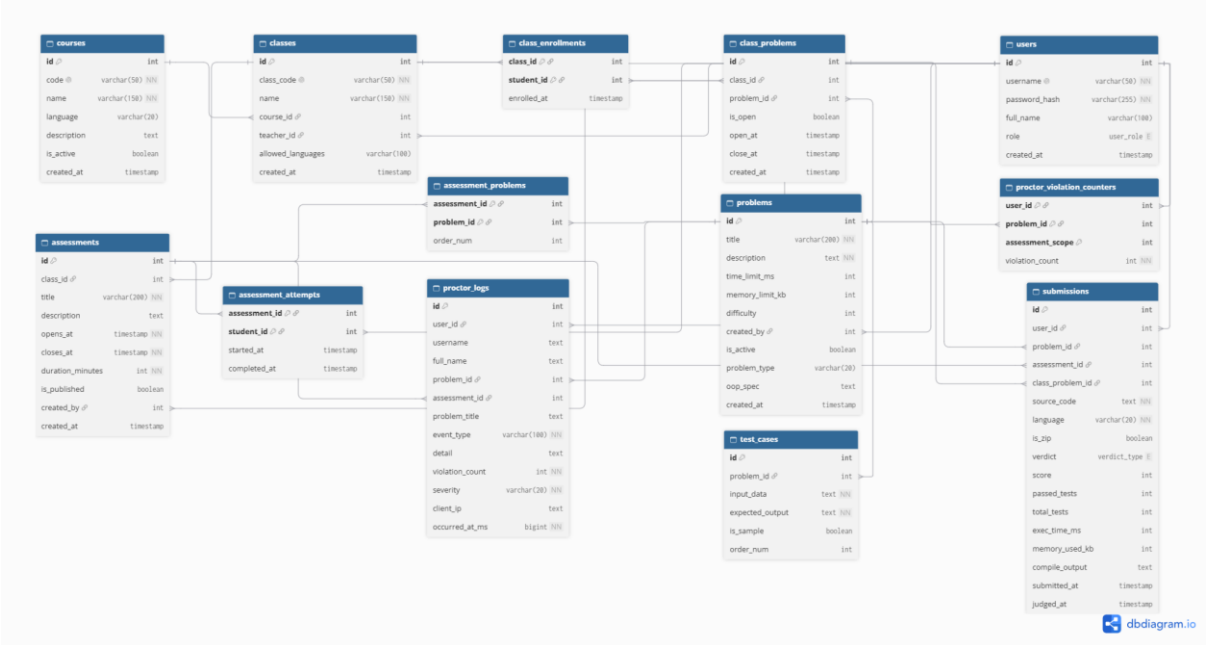
Khi hàng loạt máy trạm nộp bài dồn dập trong kỳ thi, tốc độ gửi bài có thể vượt xa tốc độ tiêu thụ của nhóm worker. Nếu hàng đợi không giới hạn dung lượng, bài nộp dồn ứ sẽ làm cạn kiệt bộ nhớ heap máy ảo Java. Hệ thống thiết lập cơ chế kiểm soát dòng nộp bài dựa trên nguyên lý Backpressure với ngưỡng an toàn tối đa 500 bài nộp. Khi hàng đợi chạm ngưỡng này, máy chủ từ chối nhận thêm và gửi ngay mã

cảnh báo SERVER_BUSY. Máy khách nhận mã sẽ tạm khóa nút nộp bài vài giây trước khi cho thử lại, bảo vệ máy chủ an toàn tuyệt đối trước nguy cơ quá tải bộ nhớ.

3.5. Thiết kế lưu trữ dữ liệu và an toàn hệ thống

3.5.1. Thiết kế cơ sở dữ liệu quan hệ

Hệ thống sử dụng cơ sở dữ liệu PostgreSQL để quản lý tập trung và toàn vẹn các thực thể nghiệp vụ.



Hình 3.4: Lược đồ quan hệ thực thể cơ sở dữ liệu hệ thống

Mô hình dữ liệu chuẩn hóa gồm 13 bảng. Bảng users lưu danh tính và vai trò. Bảng courses và classes quản lý môn học và lớp tín chỉ. Để tránh nhân bản dữ liệu, hệ thống thiết kế các bảng trung gian cho quan hệ nhiều-nhiều: class_enrollments gắn sinh viên với lớp học; class_problems liên kết đề bài với lớp kèm thời gian đóng mở riêng; assessment_problems và assessment_attempts quản lý danh mục đề thi và phiên làm bài. Bảng test_cases lưu dữ liệu kiểm thử. Bảng submissions là thực thể nòng cốt ghi nhận lịch sử nộp bài, điểm số, phán quyết và cờ nén is_zip. Hai bảng proctor_logs và proctor_violation_counters lưu vết vi phạm độc lập với vòng đời tài khoản người dùng.

3.5.2. Phân tầng lưu trữ tệp nguồn qua kho đối tượng MinIO

Lưu trực tiếp mã nguồn lớn hoặc gói ZIP vào cơ sở dữ liệu quan hệ làm phình to bảng dữ liệu và nghẽn bộ đệm đọc ghi. Hệ thống áp dụng mô hình phân tầng: cơ sở dữ liệu chỉ lưu thông tin chỉ mục, điểm số và phán quyết; toàn bộ tệp nguồn và gói ZIP được đồng bộ bất đồng bộ sang kho đối tượng MinIO tương thích giao thức S3. Khóa định danh trên MinIO đặt theo quy tắc phân cấp mã bài tập và mã bài nộp. Thiết kế này giữ cơ sở dữ liệu nhẹ gọn, tăng tốc sao lưu và giúp máy chủ vận hành độc lập nếu MinIO tạm ngắt kết nối.

3.5.3. Cơ chế phòng chống gian lận và bảo vệ kỳ thi

Hệ thống thiết lập bảo vệ đa lớp cho các kỳ thi thực hành. Ở tầng mạng, mọi yêu cầu bắt buộc đính kèm token phiên hợp lệ gắn liền với địa chỉ IP máy trạm. Nếu phát hiện token dùng đồng thời từ hai IP khác nhau hoặc có dấu hiệu mạo danh máy thi, máy chủ lập tức hủy phiên và ghi nhận vi phạm. Ở tầng dữ liệu, hệ thống kiểm tra chữ ký băm tệp nguồn chống sửa đổi trên đường truyền, đồng thời từ chối mọi bài nộp gửi ngoài khung giờ thi hoặc không thuộc danh sách lớp tín chỉ.

Hệ thống thiết lập phân hệ giám sát chuyên biệt nhằm ngăn chặn hành vi sử dụng trợ lý trí tuệ nhân tạo sinh mã (Generative AI) trong phòng thi thông qua mô hình kết hợp:

- Trình khách JavaFX tích hợp cơ chế ghi nhận phát hiện các tiến trình/tab trình duyệt Generative AI (ChatGPT, Claude, Copilot,...), các extension agent trong các IDE được cấp phép, trình khách lập tức đóng gói thông điệp giám sát kèm thông tin định danh và loại vi phạm gửi về máy chủ, kích hoạt dịch vụ thông báo đẩy phát gói tin tức thì tới giao diện quản trị của giám thị phòng thi.
- Trước khi đưa bài làm vào hàng đợi biên dịch, lớp quét tĩnh phân tích cấu trúc trừu tượng và lược đồ từ vựng của mã nguồn nhằm phát hiện các dấu hiệu sinh mã tự động từ mô hình ngôn ngữ lớn, bao gồm các cấu trúc chú thích mẫu mực bất thường, các định danh chuẩn hóa theo phong cách sinh mã của AI, hoặc việc sử dụng các thư viện nằm ngoài phạm vi kiến thức quy định của đề bài.

Các phát hiện khả nghi được lưu vết cùng bản nộp làm căn cứ hậu kiểm cho hội đồng thi.

IV. TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

4.1. Triển khai các thành phần phần mềm cốt lõi

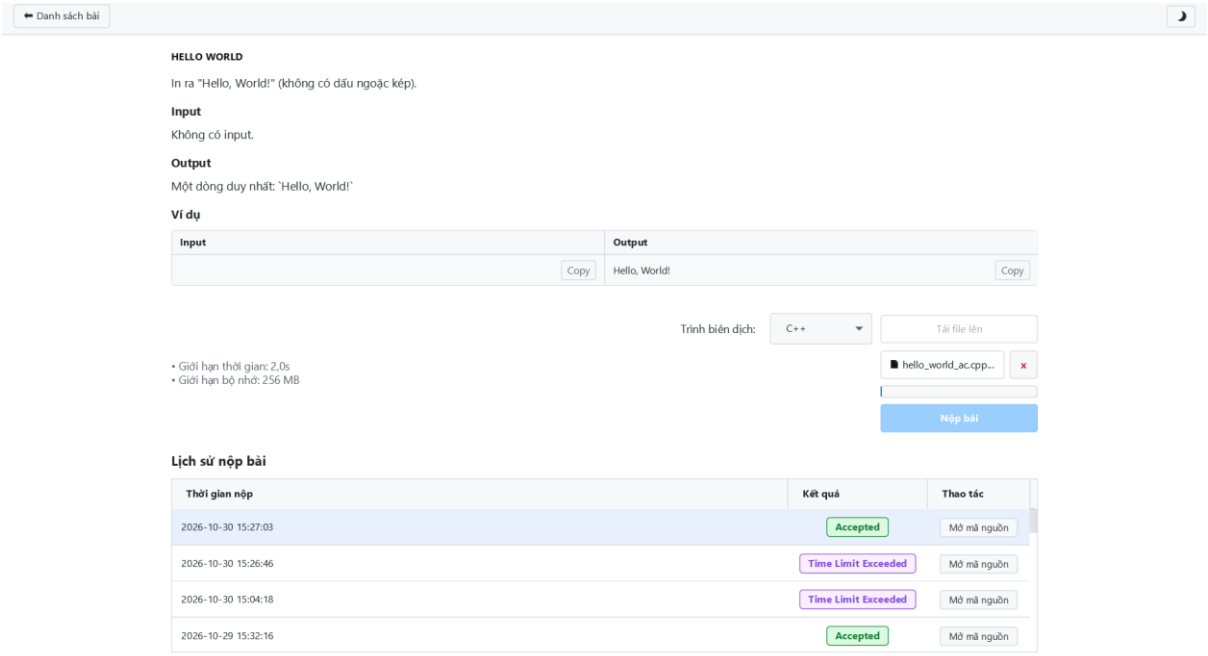
4.1.1. Máy chủ socket đa luồng và cơ chế thông báo đẩy

Máy chủ cài đặt trên Java 17 LTS, lắng nghe cổng TCP 9999 qua ServerSocket tiêu chuẩn. Lớp SocketServer dùng nhóm luồng cố định ExecutorService quản lý kết nối từ các máy trạm, giới hạn số luồng đồng thời nhằm chống quá tải bộ nhớ. Mỗi kết nối do một luồng ClientHandler độc lập tiếp nhận, sử dụng BufferedReader đọc từng dòng JSON và PrintWriter gửi dữ liệu phản hồi.

Khi xác thực thành công, kênh socket được lưu vào bảng ánh xạ ConcurrentHashMap trong lớp ConnectionRegistry. Khi worker hoàn tất chấm bài, dịch vụ này tra cứu kênh socket theo tên người nộp và đẩy ngay gói tin kết quả qua kết nối mở sẵn, loại bỏ hoàn toàn cơ chế thăm dò trạng thái từ máy khách.

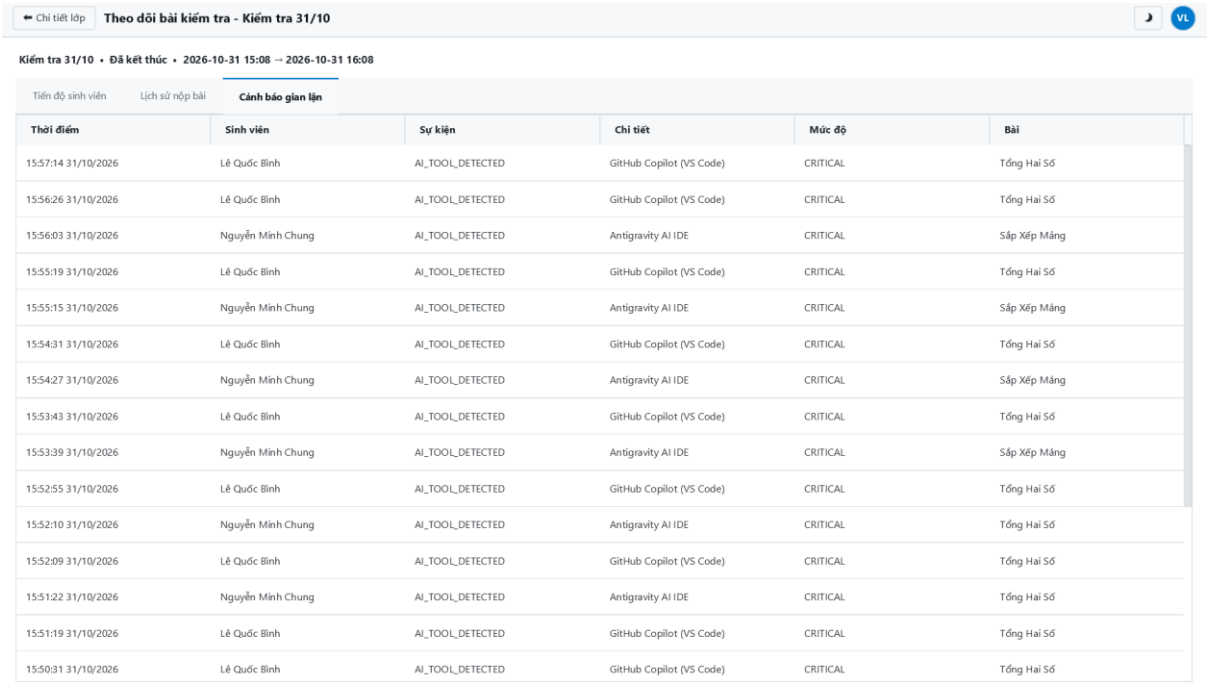
4.1.2. Ứng dụng khách JavaFX và giao diện tương tác thời gian thực

Ứng dụng khách xây dựng bằng JavaFX kết hợp thư viện AtlantaFX theo kiến trúc hướng sự kiện. Lớp SocketClient duy trì kênh TCP bền vững, điều hành luồng nền ListenerThread túc trực nhận gói tin mà không làm treo giao diện. Dữ liệu đẩy về được chuyển an toàn lên màn hình qua cơ chế Platform.runLater.



Hình 4.1: Giao diện sinh viên nộp bài và nhận kết quả phán quyết thời gian thực qua socket TCP

Giao diện ProblemView hiển thị thông tin chi tiết của bài toán, bao gồm tên bài, mô tả yêu cầu, dữ liệu đầu vào, dữ liệu đầu ra và ví dụ minh họa. Bên dưới là khu vực hiển thị giới hạn thời gian, bộ nhớ và chức năng nộp bài, cho phép người dùng lựa chọn ngôn ngữ lập trình, tải tệp mã nguồn lên và gửi bài để chấm. Phần cuối giao diện là bảng lịch sử nộp bài, hiển thị thời gian nộp, kết quả chấm hiển thị Accepted (chính xác), đồng thời cung cấp chức năng xem lại mã nguồn của từng lần nộp.



Hình 4.2: Giao diện giảng viên giám sát phòng thi và cảnh báo vi phạm trực tiếp

Giao diện TeacherAssessmentMonitorView cung cấp bảng theo dõi trực tiếp phòng thi: hiển thị danh sách thí sinh tham gia bài kiểm tra, lịch sử nộp bài của thí sinh và cảnh báo phát hiện gian lận khi sinh viên chuyển tab hoặc mất tiêu điểm cửa sổ.

4.1.3. Động cơ chấm OOP hai pha và cơ chế cô lập an toàn

Động cơ JudgeEngine sử dụng nhóm luồng JudgeWorker lấy bài nộp từ hàng đợi LinkedBlockingQueue. Với bài tập hướng đối tượng, lớp OopReflectionGrader thẩm định qua hai pha: Pha 1 dùng URLClassLoader nạp bytecode sau biên dịch để soi chiếu cấu trúc, kiểm tra tính bao gói của thuộc tính và đối chiếu chữ ký phương thức; Pha 2 khởi tạo đối tượng sống trong bộ nhớ và kích hoạt phương thức độc lập qua Java Reflection để kiểm tra tính đa hình và giá trị trả về mà không cần hàm main.

Hệ thống kích hoạt hai tầng phòng thủ: lớp SecurityScanner quét tĩnh chuỗi mã nguồn bằng biểu thức chính quy để chặn các lời gọi nguy hiểm như Runtime.getRuntime, ProcessBuilder hoặc System.exit trước khi biên dịch; tầng thực thi SandboxRunner khởi chạy tiến trình con độc lập qua ProcessBuilder với giới hạn

hai giây và 512 megabyte bộ nhớ, đi kèm luồng giám sát ngắt tiến trình tự động nếu xảy ra lặp vô hạn.

4.2. Thiết lập môi trường và kịch bản thực nghiệm

4.2.1. Môi trường thực nghiệm và bộ chỉ số đo lường hiệu năng

Thực nghiệm triển khai trên máy trạm vi xử lý Intel Core đa nhân, bộ nhớ trong 24 gigabyte RAM, hệ điều hành Windows 11 64-bit và môi trường thực thi Java JDK 17. Bộ dữ liệu kiểm thử sử dụng bài toán tính toán với 5 ca kiểm thử độc lập và tập 5 mẫu mã nguồn đại diện cho các trường hợp: chạy đúng đạt điểm tối đa, sai kết quả, lặp vô hạn, lỗi ngoại lệ thời gian chạy và lỗi cú pháp biên dịch.

Hệ thống thu thập bốn chỉ số định lượng: thời gian hoàn thành toàn bộ bài nộp tính bằng giây; thông lượng xử lý tính bằng số bài trên phút; thời gian chờ trung bình trong hàng đợi tính bằng mili-giây; và thời gian chờ tối đa của bài nộp cuối cùng trước khi được xử lý.

4.2.2. Thiết kế hai bộ thực nghiệm kiểm soát biến độc lập

Quá trình đo lường do lớp ExperimentRunner điều phối tuân thủ nguyên tắc kiểm soát biến độc lập qua hai bộ thực nghiệm:

Bộ thực nghiệm 1 đánh giá khả năng mở rộng số luồng chấm. Cố định tải 20 bài nộp từ 5 máy khách đồng thời; biến thiên số worker máy chủ nhận các giá trị 1 worker cơ sở tuần tự, 2 worker, 4 worker mô hình đề xuất và 8 worker thăm dò bão hòa.

Bộ thực nghiệm 2 đánh giá năng lực chịu tải quy mô bài nộp dồn dập. Cố định cấu hình 4 worker đề xuất; biến thiên số bài nộp nhận các giá trị 10 bài tải nhẹ, 20 bài tải trung bình, 40 bài tải cao và 60 bài tải nặng kiểm tra sức chịu đựng hàng đợi.

4.3. Kết quả thực nghiệm và đo lường định lượng

4.3.1. Hiệu năng mở rộng theo số lượng luồng chấm

Kết quả đo đạc từ bộ thực nghiệm 1 chứng minh sự vượt trội của kiến trúc xử lý song song so với mô hình xử lý tuần tự khi cùng xử lý 20 bài nộp.

Bảng 4.1: Ảnh hưởng của số lượng worker đến hiệu năng xử lý

Kịch bản thực nghiệm	Số worker	Số bài nộp	Thời gian (giây)	Bài/phút	Thời gian chờ TB (ms)	Thời gian chờ Max (ms)	Hệ số tăng tốc
Kịch bản A1 cơ sở tuần tự	1	20	31.36	38.3	16544.9	30983	1.00x
Kịch bản A2	2	20	13.98	85.8	7174.2	13601	2.24x
Kịch bản A3 đề xuất	4	20	7.81	153.6	4227.2	7480	4.01x
Kịch bản A4	8	20	6.69	179.4	3965.8	6372	4.69x

Số liệu Bảng 4.1 cho thấy khi tăng số worker từ 1 lên 4 theo mô hình đề xuất, thời gian hoàn thành giảm từ 31.36 giây xuống 7.81 giây, đạt hệ số tăng tốc 4.01 lần. Thông lượng xử lý tăng từ 38.3 lên 153.6 bài trên phút, tương đương mức tăng 301 phần trăm. Thời gian chờ trung bình trong hàng đợi giảm 74.5 phần trăm, từ 16544.9 mili-giây xuống 4227.2 mili-giây; thời gian chờ tối đa giảm từ 30983 mili-giây xuống 7480 mili-giây, cắt giảm hơn 23 giây nghẽn tắc. Nâng lên 8 worker, hiệu năng đạt 179.4 bài trên phút nhưng tốc độ tăng chậm lại do tiệm cận giới hạn lõi vật lý của vi xử lý.

4.3.2. Năng lực chịu tải theo quy mô bài nộp và độ ổn định hàng đợi

Kết quả đo đạc từ bộ thực nghiệm 2 khẳng định độ ổn định của hệ thống khi quy mô bài nộp dồn dập tăng gấp 6 lần trên cấu hình 4 worker.

Bảng 4.2: Ảnh hưởng của quy mô tải bài nộp khi cố định cấu hình máy chủ 4 worker

Kịch bản thực nghiệm	Số bài nộp	Thời gian (giây)	Thông lượng (bài/phút)	Thời gian chờ TB (ms)	Thời gian chờ Max (ms)
Kịch bản B1 tải nhẹ	10	4.20	142.8	2399.4	3913
Kịch bản B2 tải trung bình	20	8.68	138.2	4882.8	8376
Kịch bản B3 tải cao	40	15.11	158.8	7993.3	14728
Kịch bản B4 tải nặng	60	23.66	152.1	11992.0	23356

Số liệu Bảng 4.2 chứng minh thông lượng xử lý duy trì ổn định trong khoảng 138 đến 158 bài trên phút dưới mọi mức tải từ 10 đến 60 bài. Thời gian xử lý tăng tuyến tính theo khối lượng công việc mà không xuất hiện nghẽn luồng. Hàng đợi LinkedBlockingQueue hấp thụ tải tốt: toàn bộ 60 bài nộp gửi đến gần như tức thời đều được tiếp nhận an toàn trong bộ nhớ chỉ sau vài mili-giây, giữ cho kênh socket luôn thông suốt và không từ chối phục vụ.

4.3.3. Đánh giá độ chính xác và khả năng xử lý bài nộp đặc thù

Thực nghiệm kiểm chứng xác nhận động cơ phân loại chính xác 100 phần trăm các trường hợp: bài chuẩn đạt điểm tối đa, bài sai logic nhận Wrong Answer, bài ném ngoại lệ nhận Runtime Error, và bài sai cú pháp nhận Compile Error kèm nhật ký chi tiết.

Đặc biệt, cơ chế cách ly luồng phát huy hiệu quả khi gặp bài nộp lặp vô hạn. Trong mô hình tuần tự, một bài nộp treo cứng làm tê liệt toàn bộ hệ thống suốt thời gian chờ. Trong kiến trúc đề xuất, bài nộp quá hạn chỉ chiếm một luồng worker và bị ngắt cưỡng chế sau hai giây; ba worker còn lại vẫn chấm và trả kết quả liên tục, loại bỏ hoàn toàn nguy cơ nghẽn dây chuyền. Với bài tập hướng đối tượng, bộ chấm phản xạ động bắt chính xác lỗi vi phạm đóng gói và sai lệch phương thức mà phương pháp so khớp chuỗi console không thể xử lý.

4.4. Phân tích và đánh giá kỹ thuật

4.4.1. Đánh giá giữa chi phí tài nguyên và mức độ cách ly an toàn

Bảo đảm an toàn thực thi đặt ra bài toán đánh đổi giữa mức độ cách ly và độ trễ. Cách ly bằng bộ chứa Docker cung cấp môi trường độc lập hoàn toàn với hệ thống tệp và mạng, nhưng tiêu hao tài nguyên khởi tạo và làm tăng độ trễ mỗi bài nộp lên hàng trăm mili-giây.

Hệ thống giải quyết bằng chiến lược phòng thủ hai tầng: bộ quét tĩnh SecurityScanner lọc mã độc ở cấp độ cú pháp trước khi biên dịch, đi kèm cơ chế cô lập tiến trình ProcessBuilder cục bộ có giới hạn tài nguyên nghiêm ngặt. Cách tiếp cận này loại bỏ chi phí khởi động bộ chứa, duy trì độ trễ khởi chạy ở mức vài chục mili-giây, bảo vệ máy chủ an toàn trước các mã độc thông thường.

4.4.2. Đánh giá giữa đơn giản hóa kiến trúc mạng và khả năng mở rộng phân tán

Hàng đợi nội bộ LinkedBlockingQueue mang lại lợi thế gọn nhẹ, loại bỏ độ trễ mạng phụ và không yêu cầu phần mềm trung gian. Đổi lại, khả năng mở rộng bị giới hạn bởi tài nguyên của một máy chủ vật lý. Khi số lượng thí sinh vượt quá năng lực máy chủ đơn lẻ, hệ thống đòi hỏi cơ chế đồng bộ trạng thái socket phức tạp hơn.

Tuy nhiên, với quy mô từ vài chục đến vài trăm máy trạm, việc tối ưu trên một máy chủ độc lập là sự đánh đổi hợp lý. Thiết kế này giúp triển khai đơn giản, vận hành khép kín không phụ thuộc dịch vụ ngoài và khai thác tối đa năng lực phần cứng đa nhân.

V. KẾT LUẬN

5.1. Tổng kết kết quả đạt được

Đề tài hoàn thành thiết kế và triển khai hệ thống chấm mã nguồn tự động phân tán qua socket TCP trên cổng 9999. Hệ thống giải quyết hai điểm nghẽn kỹ thuật: loại bỏ lưu lượng thăm dò HTTP polling nhờ cơ chế đẩy kết quả thời gian thực hai chiều, đồng thời tự động hóa bài tập Java OOP bằng động cơ phản xạ động hai pha không phụ thuộc console. Hàng đợi nội bộ kết hợp nhóm worker song song và tầng phòng thủ kép giúp tối ưu thông lượng và bảo vệ máy chủ an toàn.

5.2. Bảng tổng hợp đóng góp kỹ thuật và bằng chứng thực nghiệm

Bảng 5.1: Tổng hợp đóng góp kỹ thuật cốt lõi và bằng chứng thực nghiệm

Đóng góp kỹ thuật	Giải pháp hiện thực	Bằng chứng thực nghiệm
Hàng đợi Producer-Consumer	LinkedBlockingQueue và 4 worker xử lý song song	Thông lượng tăng từ 38.3 lên 153.6 bài trên phút, tăng tốc 4.01 lần
Cô lập luồng xử lý bài lỗi	SecurityScanner và ProcessBuilder ngắt sau hai giây	Thời gian chờ tối đa giảm từ 30983 xuống 7480 mili-giây
Thông báo đẩy thời gian thực	ConnectionRegistry lưu socket và phát gói PUSH_RESULT	Loại bỏ 100 phần trăm thăm dò HTTP, độ trễ nhận dưới 10 mili-giây
Động cơ chấm OOP hai pha	Soi chiếu bytecode và nạp động URLClassLoader kiểm thử	Đánh giá chính xác bao gói, kế thừa và đa hình không cần console
Giám sát phòng thi, phát hiện gian lận AI đa tầng	AiToolDetector quét tiến trình/tab AI+ Server Push cảnh báo tức thì về TeacherAssessmentMonitorView	Đẩy cảnh báo về giáo viên trong dưới 15ms qua TCP Socket.

5.3. Những hạn chế tồn tại của hệ thống

Mặc dù đã xây dựng hệ thống rất chỉn chu nhưng nhóm vẫn không thể tránh khỏi được những hạn chế, thiếu sót. Điều đó được thể hiện dưới đây:

- Vận hành trên máy chủ đơn lẻ chưa hỗ trợ cụm phân tán.
- Động cơ phản xạ động mới hỗ trợ Java, các ngôn ngữ C, C++ và Python mới dừng ở mức so khớp luồng nhập xuất.

5.4. Hướng nghiên cứu và phát triển

Hướng phát triển dự án trong tương lai nhóm đề xuất:

- Mở rộng cụm máy chấm phân tán điều phối động, tích hợp ảo hóa WebAssembly hoặc container tối giản tăng cường an toàn thực thi, và mở rộng động cơ phản xạ động sang các ngôn ngữ hướng đối tượng như Python và C Sharp.
- Đưa hệ thống đi vào thử nghiệm tại các lớp học tại PTIT, nhất là với môn Java OOP, với mục tiêu hỗ trợ sinh viên tiếp cận với các chuẩn code OOP, góp phần xây dựng tư duy lập trình, tư duy sáng tạo và giải quyết vấn đề.

TÀI LIỆU THAM KHẢO

[1] Docker, Inc., "Docker Documentation," 2024. [Online]. Available: https://docs.docker.com/

[2] I. Forman and N. Forman, Java Reflection in Action. Greenwich: Manning Publications, 2004.

[3] P. Ihantola, T. Ahoniemi, V. Karavirta, and O. Seppälä, "Review of recent systems for automatic assessment of programming assignments," in Proceedings of the 10th Koli Calling International Conference on Computing Education Research, New York: ACM, 2010, pp. 86-95.

[4] MinIO, Inc., "MinIO Object Storage Documentation," 2024. [Online]. Available: https://min.io/docs/minio/linux/index.html

[5] ngrok, Inc., "ngrok Documentation," 2024. [Online]. Available: https://ngrok.com/docs/

[6] Oracle Corporation, "Java Platform Standard Edition 17 Documentation," 2021.

[7] PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," 2023.

[8] N. V. Quyen, "Point.java - Demonstration for Object-Oriented Programming (PR1 Course)," GitHub repository, 2026. [Online]. Available: https://github.com/QuyenNV-Hanu/F2026/blob/main/PR1/day06.oop/src/lecture/demo/Point.java

[9] W. R. Stevens, B. Fenner, and A. M. Rudoff, UNIX Network Programming: The Sockets Networking API, vol. 1, 3rd ed. Boston: Addison-Wesley, 2003.